

Zedral • M4 — OEE & Downtime Intelligence

Developer Handover • v1 for build • 2026-06-19

Table of Contents

Zedral • M4 • OEE & Downtime Intelligence — Developer Handover

Status: v1 for build (capture-agnostic OEE + Six-Big-Losses tree; PLC auto-capture & prediction = interfaces only). **Companion:** OEE deep-plan (../M4_OEE_Downtime_DeepPlan.md) + 10 flowcharts (../M4_diagram_*.mermaid) + runnable M4_schema.sql. *Architecture diagrams below are shown as Mermaid source; the same diagrams render from the standalone .mermaid files.*

Zedral • M4 OEE & Downtime Intelligence — Developer Handover

00 • Master Index, Architecture, Stack & Conventions

Audience: engineering team • **Status:** v1 for build (capture-agnostic OEE + Six-Big-Losses tree; PLC auto-capture & prediction = interfaces only) • **Date:** 2026-06-19

This is the build-ready specification set for **M4 • OEE & Downtime Intelligence**, the platform's loss-accounting module (the foundation layers — Data Lake, Canonical Model, Manifold, Unified Intelligence, Monitoring, Reporting, Financial Impact, Platform/Security — are specified in ../../Dev_Handover/; sibling modules M2 Maintenance and M3 Planning are in ../../M2_Maintenance_Intelligence/ and ../../M3_Shopfloor_Planning_Management/). Read this index first: module scope, the architecture, which **platform decisions (D1–D11)** bind M4, the **stack** M4 inherits, M4-specific conventions, and the **build roadmap**. The *why* behind every spec is the OEE deep-plan one level up: ../M4_OEE_Downtime_DeepPlan.md.

1. Document set & reading order

#	Spec	Build priority
00	This index — architecture, stack, conventions, roadmap	Read first

01	Data Model & Schema — new derived entities, DDL dictionary (M4_schema.sql), the no-double-count anchor	Phase M4-0
02	OEE Calculation & Time Model — $A \times P \times Q + TEEP$, ISO 22400 time model, roll-up rule, ideal-cycle	Phase M4-1→2
03	State & Downtime Capture — state model, interval resolver, micro-stop/debounce, M1 bootstrap, PLC future	Phase M4-1
04	Loss Taxonomy & Reason Codes — Six Big Losses, 3-layer reason tree, Pareto/“All Other” governance	Phase M4-2
05	Aggregation, Events & Pipeline — streaming + batch rollups, replay/recompute, domain events	Phase M4-1→2
06	KPIs, Analytics & Financial Impact — formulas + SQL, registry, M4↔M2 / M4↔M3 reconciliation	Phase M4-2
07	APIs, Events & Integration — REST/OpenAPI, OPC-UA/MQTT collector contract, MVP migration	Phase M4-1→4

08	Prediction / ML — Future-Scope Interfaces — loss/micro-stop/sp eed-loss hooks, readiness gate (no v1 build)	Phase M4-5 (gated)
09	Security, RBAC, NFRs & Acceptance — roles, multi- tenancy, NFRs, test plan, acceptance	Cross-cutting
10	Glossary — OEE & loss terms	Reference
—	M4_schema.sql	Runnable PostgreSQL DDL (paired with 01)

Companion material: the OEE deep-plan (../M4_OEE_Downtime_DeepPlan.md) + 10 flowcharts (../M4_diagram_*.mermaid). Platform specs referenced throughout live in ../Dev_Handover/.

2. Module scope & non-goals

In scope (v1): OEE reference & policy (state model, micro-stop thresholds, ideal-cycle/rated-speed reference, reason→loss→component mapping, targets); the **State Interval Resolver** (debounce, close, classify); **OEE = A×P×Q + TEEP/OPE** to ISO 22400; the **Six-Big-Losses** loss tree + top-loss Pareto (by minutes and cost); financial pricing of every loss; live OEE/state board + micro-stop/speed alerts; **v1 capture by bootstrapping M1** (manual stoppages, counts, shift_log) with ingestion from a historian via Manifold as an alternative on-ramp.

Future scope (interfaces only, build gated — D9): automatic PLC/OPC-UA/MQTT **state detection** (spec 03/07 — collector contract specified, switch-on when a client connects PLCs); **loss / micro-stop / speed-loss prediction** (spec 08), prescriptive recommendations. v1 ships the data hooks so both are a later switch-on, not a re-model.

Non-goals (neighbouring modules/systems): data capture itself (M1 owns first-party capture; M4 consumes the canonical events); maintenance/reliability (M2 owns failure mode, MTBF, MTTR on the *shared* DowntimeEvent); plan attainment (M3 owns “did we run the plan?” on the *shared* ProductionCount); defect disposition & quality holds (M6); yield economics (M5); the financial ledger (M4 *feeds* Financial Impact, doesn’t own cost accounting).

3. System context

```
flowchart LR
  subgraph SRC["Events in (capture-agnostic)"]
    M1["M1 first-party: stoppages, prod * counts, shift_log (v1)"]
    PLC["PLC/OPC-UA/MQTT collector (future switch-on)"]
    HIST["Existing OEE/MES historian via Manifold"]
    SIB["M2 failure class · M3 plan targets"]
  end
  subgraph M4SVC["M4 services"]
    REF["Reference/Policy svc (state model · ideal cycle · reason map · thresholds)"]
    RES["State Interval Resolver svc"]
    CALC["OEE Calc svc (A×P×Q + TEEP)"]
    LOSS["Loss-Attribution svc (Six Big Losses + Pareto)"]
    KPI["KPI/Financial svc"]
    PRED["Prediction (future, stub)"]
  end
  CANON["canon.* (Event Spine + oee.* schema)"]
  SERVE["Serving zone / KPI mart / Redis live cache"]
  subgraph CONS["Consumers"]
    UIL["Unified Intelligence"]
    MON["Operational Monitoring"]
    FIN["Financial Impact"]
    REP["Reporting"]
    SIBC["M2 / M3 / M5 / M6 / M7"]
  end
  M1 --> RES
  PLC -.future.-> RES
  HIST --> RES
  SIB --> REF
  REF --> RES --> CALC --> LOSS --> KPI
  REF --> CALC
  PRED -.future.-> LOSS
  RES & CALC & LOSS & KPI --> CANON --> SERVE
  SERVE --> UIL & MON & FIN & REP & SIBC
```

Golden rule (inherited): M4 reads canonical data through the Serving zone and writes its derived facts back through the canonical write-back API + oee.* schema. It never reads another module's database and never re-records a canonical event. A downtime is **one** canon.event DowntimeEvent — M2 classifies it (failure), M4 aggregates it (Availability). A produced quantity is **one** canon.event ProductionCount — M3 contextualises it (attainment), M4 aggregates it (Performance/Quality).

4. Binding decisions that constrain M4 (subset of D1-D11)

#	Decision	M4 build constraint
D1	Hybrid-ready, per-client	The state resolver + live OEE tile + edge collector must run on-prem edge (plants are often

		disconnected); containerized, no cloud-only calls.
D2	Near-real-time (minutes), tunable	Live OEE tile & micro-stop/speed alerts refresh in minutes (per-tenant); the rollup mart is batch.
D4	Excel/CSV + generic DB connector first	M1 bootstrap is the first ingest path; OPC-UA/MQTT collector + historian connectors are fast-follows (spec 07).
D6	“Standard” v1 KPI tier	OEE/A/P/Q/TEEP + Six Big Losses + Pareto is the Standard tier; advanced loss analytics deepen later.
D7	Cost-rates client-owned, effective-dated, “rate as of”	Downtime/speed/quality loss pricing resolves the canon.cost_rate effective at the event date , never “today’s” rate.
D8	Logical tenant isolation default	tenant_id on every oee.* row; all queries tenant-scoped at the data-access layer.
D9	Rule-based now, ML later	OEE/loss is deterministic in v1; prediction (spec 08) and auto-capture detection logic are reserved, gated by readiness.
D11	Sector-neutral, exercise extension namespace	State/loss/OEE model is generic; Hero Steels is a <i>seed template</i> , not the schema. Client specifics live in ext JSONB. M4 is the cleanest cross-sector validator.

Full D1–D11 text: ../../Dev_Handover/00_INDEX_and_Architecture.md §3.

5. Technology stack (inherited + M4-relevant)

M4 adds **no new platform technology** (except the future edge collector); it uses the stack ratified in the platform handover.

Concern	Tech	M4 use
Canonical OLTP	PostgreSQL 15+ (oee schema)	all M4 reference + derived tables (M4_schema.sql)
KPI mart	ClickHouse / Postgres columnar	oee_interval rollups for UIL/Reporting
Stream processing	Kafka/Redpanda + Flink	state-interval resolution + live OEE off the streaming path
Scheduler / batch	Apache Airflow	shift/day OEE rollup, loss attribution, Pareto, readiness jobs
Live cache	Redis	live OEE tile, current state, micro-stop/speed counters (Monitoring)

API	REST + OpenAPI , gateway, OIDC	spec 07 endpoints
Auth/RBAC	Keycloak (OIDC)	M4 roles extend M1 (spec 09)
Backend	Python 3.11 + FastAPI	M4 services
Frontend	React + TS (PWA)	OEE board, loss Pareto, config; reuse M1 capture patterns
Edge collector (future)	OPC-UA / MQTT client on edge gateway	reserved for spec 03/07; not built in v1
Prediction (later)	MLflow + shared M2 OSA-CBM substrate	reserved for spec 08, not built in v1

6. M4 build roadmap (epics)

flowchart TD

M0["M4-0 · Foundations
oee schema · state model · ideal-cycle ref · reason-to-loss map · oee_config · RBAC · repoint MVP onto canon.*"]

M1x["M4-1 · State & Availability (closest to M1)
state interval resolver · debounce/micro-stop · downtime classify · Availability from M1 bootstrap"]

M2x["M4-2 · Full OEE + loss tree
 $A \times P \times Q$ + TEEP · Six-Big-Losses attribution · top-loss Pareto · M4↔M2/M3 reconciliation tests"]

M3x["M4-3 · KPIs · Financial · Monitoring
registry publish · priced losses · live OEE tile + alerts · reporting pack"]

M4x["M4-4 · Auto-capture (switch-on)
OPC-UA/MQTT collector · debounce at edge · operator reason-prompt · Fidelity gate rises"]

M5x["M4-5 · Prediction (GATED on data + D9)
loss/micro-stop/speed-loss models · shares M2 OSA-CBM · prescriptive later"]

M0 --> M1x --> M2x --> M3x --> M4x --> M5x

Maps onto the platform roadmap: M4-0→3 ride Phase 2–3 (serve/synthesize); M4-4 rides Phase 4 (connectors/edge); M4-5 rides Phase 5 (ML, data-gated). The **on-ramp order** (M1-bootstrap first) front-loads the cheapest value — M4 produces a real OEE on day one with no new hardware.

7. Non-functional baseline (delta over platform §7)

M4 inherits the platform NFR baseline (multi-tenant, hybrid, minutes-latency, 99.5% serving, OIDC, audit, OTel). Module-specific targets (spec 09): the live OEE tile reflects a new state within the D2 latency budget; the state resolver is **idempotent and replayable** (re-deriving an interval from corrected events yields identical OEE — no manual patching); rollups **reconcile with M2 (downtime minutes) and M3 (counts) to the unit/minute**; all definition changes (ideal cycle, threshold, planned-time convention) are effective-dated

and audit-logged so trends never silently break; OEE numbers carry a **fidelity/confidence flag** (Principle 7).

8. Open items (carry into build)

- **Planned-time convention** default (exclude from PPT vs inside Availability) — specs 02/03 (oee_config.planned_time_convention).
- **Micro-stop threshold** default (60 vs 120 s) — spec 03 (oee_config.micro_stop_threshold_sec).
- **Ideal-cycle source of truth** when nameplate/demonstrated/plan all exist — spec 02 (ideal_cycle.source).
- **TEEP exposure** default on/off — spec 06.
- **First connector** after M1 bootstrap (OPC-UA generic vs named OEE/MES) — spec 07.
- **MVP reconciliation tolerance** (e.g. ≤ 0.5 OEE pts / 30-day back-test) — spec 07/09 acceptance.
- **Prediction data gate** (labelled-loss-history threshold; share with M2 PdM gate) — spec 08.
- **D11**: use M4 on a discrete client as the cleanest cross-sector canonical validation.

Next: specs 01–10. The compiled Word/PDF package combines all of these into one M4 Developer Handover document.

M4 • 01 • Data Model & Schema

Pairs with: M4_schema.sql · **Deep-plan:** \$4, \$11 · **Build phase:** M4-0

1. Design stance

M4 is, by design, a **thin-write / heavy-read** module. The canonical model already scaffolded the OEE primitives — StateModel, ReasonCode, LossCategory, and the Event-Spine rows StateEvent / DowntimeEvent / ProductionCount (doc 01 \$10) — *for M4*. M4 **brings them to life** and adds only a **derived layer** on top: an ideal-cycle reference, per-tenant policy, resolved state intervals, computed OEE intervals, loss attribution, Pareto, readiness. It introduces **no new canonical event types** and **never re-records** an event.

2. New entities (additive, oee schema)

Entity	Table	Role
OEE policy	oee.oee_config	per-tenant definitions: micro-stop threshold, planned-time convention, ideal-cycle default, TEEP toggle
Work-unit config	oee.work_unit_config	per-asset overrides +

Ideal cycle / rated speed	oee.ideal_cycle	capture fidelity (drives confidence) the Performance denominator, keyed material × work-unit, versioned, calibrated-flag
OEE target	oee.oee_target	target A/P/Q/OEE per asset/period
State interval	oee.state_interval	debounced, closed intervals derived from canon.state_event
OEE interval	oee.oee_interval	computed A/P/Q + time/count quantities per asset × bucket
Loss attribution	oee.loss_attribution	Six-Big-Losses leaves: minutes/units/cost by reason
Top loss	oee.top_loss	Pareto cache (by minutes and by cost)
Policy change log	oee.policy_change_log	effective-dated audit of definition changes
Readiness snapshot	oee.readiness_snapshot	Required/Quality/ Fidelity/History gates
Loss prediction (stub)	oee.loss_prediction	future scope (spec 08), unbuilt in v1

3. Conventions (inherited from canonical DDL spec 02 §5)

lower_snake_case; surrogate BIGINT GENERATED BY DEFAULT AS IDENTITY PKs; UUID tenant_id on every row (D8); TIMESTAMPTZ in UTC; units embedded in column names (*_min, *_sec, *_per_hr); JSONB ext extension namespace for client/sector specifics (D11).

4. Referencing rules (the no-double-count anchor)

- **Hard FK only to canon.*** — equipment_node, shift, reason_code, loss_category, cost_rate, and event (the Event Spine).
- **Soft references to other module schemas** (e.g. ops.material in M3) are plain BIGINT *_ref with **no cross-schema FK** — so canon/M4 never depend on M3.
- state_event_ref, downtime_event_ref, count linkage all point back to canon.event(event_id) — M4 **annotates** the shared event, never copies it. downtime_event_ref is the exact column M2 also keys on; that shared key is what makes the M4←M2 no-double-count rule mechanical (deep-plan §12).

5. The quantity-not-ratio rule (critical)

oee.oee_interval stores **PPT, operating time, total/good counts** as quantities, not only the A/P/Q ratios. Roll-ups (oee.v_oee_rollup) **SUM the quantities, then divide** — never average sub-interval percentages. This structurally forbids the most common OEE error (deep-plan §4). Reviewers: reject any rollup query that AVG(oee).

6. Indexing & partitioning

Time-series tables (state_interval, oee_interval, loss_attribution) are tenant+asset+time indexed and **partitioned by month** in production (inherits the platform partitioning pattern, doc 01). oee_interval is uniquely keyed (tenant_id, work_unit_ref, grain, bucket_start) for idempotent upsert on recompute.

7. Acceptance (MUST)

- M-01 Every oee.* table carries tenant_id; no query crosses tenants.
- M-02 No oee.* table has a hard FK into any non-canon schema.
- M-03 oee_interval persists time & count quantities; the rollup view divides sums (no AVG of %).
- M-04 A corrected reason/state re-derives the affected state_interval/oee_interval/loss_attribution deterministically (idempotent upsert).
- M-05 loss_attribution.downtime_event_ref equals the canon.event_id M2 reads for the same stop (shared key, verified by reconciliation test in spec 06).

M4 • 02 • OEE Calculation & Time Model

Deep-plan: §4 • **Build phase:** M4-1→2 • **Standard:** ISO 22400-2

1. The three factors (ISO 22400-2 — the platform's ratified definitions, doc 01 §9)

Availability = Operating Time ÷ Planned Production Time

Performance = (Ideal Cycle Time × Total Count) ÷ Operating Time

Quality = Good Count ÷ Total Count

OEE = Availability × Performance × Quality

All three are computed from oee.oee_interval quantities. No definition lives in app code that the shared KPI registry doesn't also hold (M2/M3/UIL must read identically).

2. Equipment time model (the denominators)

Bucket	Definition	Column
Calendar Time	24×7 wall clock	calendar_time_min
Planned Production Time (PPT)	scheduled minus planned non-production	planned_production_min ← canon.shift
Operating Time	PPT minus all downtime	operating_time_min − Σ

		running state_interval
Net Operating Time	Ideal Cycle × Total Count	derived
Valuable Time	Ideal Cycle × Good Count	derived

Planned-time convention (config). oee_config.planned_time_convention selects whether planned stops (breaks, PM, no-demand) are **removed from PPT** (EXCLUDE_FROM_PPT) or **counted as planned downtime inside Availability** (INSIDE_AVAILABILITY). Both are ISO-valid; pick the house default, allow tenant override. The choice must be stamped on every interval so a trend is never silently redefined.

3. Variants

TEEP = OEE × Utilisation, Utilisation = PPT ÷ Calendar Time

Loading = PPT ÷ Calendar Time

OPE/OAE = OEE with "ideal" = demonstrated-best rate, not theoretical

TEEP exposes unscheduled capacity (the 24×7 view). oee_config.teep_enabled toggles default exposure.

4. The ideal-cycle problem (make-or-break)

Performance is only as honest as oee.ideal_cycle. Keyed **material × work-unit**, versioned, with source ∈ {NAMEPLATE, DEMONSTRATED_BEST, M3_PLAN_TARGET, MANUAL} and an is_calibrated flag. - **DEMONSTRATED_BEST** (default): 95th-percentile sustained observed rate over a trailing window (the OPE-pragmatic value) — computed by a batch job, not hand-entered. - Any interval whose Performance rests on is_calibrated = false is flagged by the **Fidelity gate** (spec 09/\$11) and shown as *indicative*, never as a confident number. - Steel: rated speed differs by gauge × width, so the key includes grade_ref; a thin-narrow coil and a thick-wide coil are different denominators.

5. Roll-up rule (reference SQL)

Compute at the leaf grain, then aggregate by **summing quantities and dividing** (see oee.v_oe_rollback):

```

SELECT work_unit_ref,
       SUM(operating_time_min) / NULLIF(SUM(planned_production_min),0)
AS availability,
       (SUM(total_count) * MAX(ideal_cycle_time_sec)/60.0)
       / NULLIF(SUM(operating_time_min),0) AS
performance,
       SUM(good_count) / NULLIF(SUM(total_count),0) AS quality
FROM oee.oe_interval
WHERE bucket_start >= :from AND bucket_start < :to
GROUP BY work_unit_ref;
```

Never AVG(oe). OEE of a line = recompute from summed child quantities (mixed ideal cycles are handled by summing Net Operating Time, not by averaging Performance).

6. Acceptance (MUST)

- C-01 A/P/Q/OEE match a hand-worked ISO 22400 example to 4 dp on a fixture dataset.
- C-02 Rollup over N intervals equals the single-interval OEE computed from summed quantities (no averaging drift).
- C-03 Switching planned_time_convention re-derives Availability deterministically and is logged in policy_change_log.
- C-04 An uncalibrated ideal cycle marks every dependent OEE as fidelity != AUTOMATIC indicative and raises the Fidelity gate.
- C-05 TEEP = OEE × (PPT ÷ Calendar) holds on the fixture.

M4 · 03 · State & Downtime Capture

Deep-plan: \$5 · **Build phase:** M4-1 (manual) → M4-4 (auto, switch-on)

1. The state model

Controlled vocabulary (oee.machine_state), each state carrying is_planned and counts_as (OEE component): | State | Running | is_planned | counts_as | Big Loss | |—|—| —|—|—| | RUNNING | yes | — | NONE | — | | MINOR_STOP | no (short) | false | PERFORMANCE | L3 | | BREAKDOWN | no | false | AVAILABILITY | L1 | | SETUP | no | true | AVAILABILITY | L2 | | PLANNED_STOP | no | true | (per convention) | planned | | NOT_SCHEDULED | no | true | NONE (TEEP only) | — |

2. State Interval Resolver (the core v1 job)

Input: raw canon.state_event transitions (from any source). Output: closed, gap-free oee.state_interval rows. Steps: **(1)** order events per work-unit by ts; **(2)** close each interval at the next transition; **(3) debounce** — collapse flapping below a debounce window; **(4)** apply the **micro-stop threshold** (work_unit_config.micro_stop_threshold_sec ?? oee_config default; 60–120 s) — a stop *below* it becomes MINOR_STOP (is_micro_stop=true, Performance/L3), *above* it stays BREAKDOWN/SETUP (Availability); **(5)** stamp counts_as; **(6)** link state_event_ref (and downtime_event_ref when the stop is a downtime shared with M2). The resolver is **idempotent and replayable**: re-running over corrected events yields identical intervals.

The micro-stop threshold is the **A/P seam** — it must be explicit config, logged on change, and identical across recompute, so the Availability/Performance split is reproducible rather than an artefact of capture.

3. v1 — M1 bootstrap (no new hardware)

Map M1 capture onto the canonical contract: | M1 source | → canonical | M4 use | |—|—|—| | txn.stoppage_entry (category OPN/ELECT/MECH/UTILITY/POWER/PLANNED/OTHER + start/end + reason) | DowntimeEvent + StateEvent | Availability + reason attribution | | prod_* counts (good/scrap, MT) | ProductionCount | Performance + Quality | | shift_log

(date, shift, process) | Shift / PPT | Availability denominator | M1 stoppage categories seed the Layer-1 reason tree (spec 04); shift_log is the time spine. OEE is produced per process/shift/coil immediately.

4. Future — automatic detection (collector contract, switch-on)

Architected now, built when a client connects PLCs (D9). Edge collector taps a **PLC run/cycle bit, OPC-UA tag, MQTT topic, or clamp-on current sensor**, applies debounce + micro-stop threshold **at the edge**, and emits the **same** canon.state_events. Operators are then prompted to reason-code only **unexplained** stops. Target: ≥98% run/idle accuracy, <0.5% false-positive/hr. Nothing downstream changes; the work_unit_config.capture_fidelity flips MANUAL → AUTOMATIC, micro_stops_observed → true, and the **Fidelity gate** (and OEE confidence) rises. Collector API in spec 07 §4.

5. Why fidelity matters (honest OEE)

Manual capture's blind spot is **sub-threshold micro-stops** — the largest single source of OEE under-reporting; automatic capture typically reveals **30–50% more downtime**. In v1 (manual), M4 reports micro-stop capture as a **readiness gap** (Fidelity gate passed=false), never as zero, so the OEE is labelled *indicative* rather than falsely precise.

6. Acceptance (MUST)

- S-01 Resolver produces gap-free, non-overlapping intervals per work-unit (no time uncounted, none double-counted).
- S-02 A stop equal to the threshold $\pm \epsilon$ classifies deterministically and identically on recompute.
- S-03 Changing the threshold is logged and re-derives the A/P split without manual edits.
- S-04 M1 bootstrap yields a non-null OEE for a Hero-Steels process/shift fixture with no PLC data.
- S-05 (auto path) the same fixture replayed via simulated PLC events yields identical Availability to ground truth within tolerance, and micro_stops_observed=true.

M4 • 04 • Loss Taxonomy & Reason Codes

Deep-plan: §6 • Build phase: M4-2

1. The loss tree (the product)

Every lost minute/unit decomposes into exactly one of the **Six Big Losses**, each mapped to one OEE component: | OEE component | Big Loss (oee.big_loss) | Carrier | |—|—|—| | Availability | L1_BREAKDOWN (unplanned) | DowntimeEvent is_planned=false | | Availability | L2_SETUP (changeover/adjust) | DowntimeEvent is_planned=true | | Performance | L3_MINOR_STOP (idling/small stops) | state_interval is_micro_stop | | Performance | L4_REDUCED_SPEED | count vs ideal-cycle gap | | Quality | L5_DEFECT

(steady-state) | ProductionCount scrap/rework | | Quality | L6_STARTUP_YIELD | run-up/transition window |

The tree **closes**: Σ Availability-loss minutes = PPT – Operating; Σ Performance loss = Operating – Net; Σ Quality loss = Net – Valuable. A reconciliation test asserts the leaves sum to the factor gaps (spec 06).

2. The 3-layer reason hierarchy (operator-first)

- **Layer 1 — category** (6–8 max): Mechanical, Electrical, Material, Changeover, Operator/Process, Utility, Quality, Planned. Selectable in 2–3 s; minimum-viable data even if the shift ends here.
- **Layer 2 — equipment/subsystem** (15–25 per category, filtered by L1): e.g. Mechanical → roll / bearing / drive / hydraulics.
- **Layer 3 — root-cause detail**: optional for stops <15 min, required ≥15 min. Rules enforced: **no level shows >15–20 options**; reasons describe the **observed symptom, not the diagnosis** (root-cause is M2’s job on the same event); **6–12 active reasons per asset** is the target.

3. Canonical mapping

Each `canon.reason_code` carries `loss_category_ref` (→ Six Big Losses) and `oee_component` (A/P/Q). Seeded by the **OEE sector template** (Manifold), tunable per tenant. This mapping is what lets a breakdown and a changeover be told apart inside Availability and what makes the tree close.

4. “All Other” governance

Exactly **one** unclassified reason exists. A scheduled job checks its share; when it enters the **top-loss Pareto or exceeds ~10%**, M4 raises an **“Other is now a top loss — refine it”** prompt and flags `top_loss.is_other_bucket=true`. The CI engineer then splits it into specific codes with the crew. This keeps the Pareto actionable and prevents the “miscellaneous black hole.”

5. Pareto (by minutes AND by cost)

`oee.top_loss` is computed per scope (asset/line/plant) and period for **two rank_basis values: MINUTES and COST** — because the biggest *time* loss and the most *expensive* loss are often different events, and the executive wants the second. Cost uses `loss_attribution.lost_cost` (priced via `canon.cost_rate`, D7).

6. Acceptance (MUST)

- L-01 Every active reason maps to exactly one loss category and one OEE component.
- L-02 Leaf losses reconcile to the A/P/Q factor gaps within rounding (closure test).
- L-03 No reason level presents >20 options; L1 has ≤8 categories.
- L-04 The “All Other” share is monitored; exceeding threshold raises the refine prompt and sets `is_other_bucket`.

- L-05 Pareto exists for both MINUTES and COST and ranks consistently with loss_attribution.

M4 · 05 · Aggregation, Events & Pipeline

Deep-plan: \$3, \$9 · **Build phase:** M4-1→2

1. Two paths, one truth

- **Streaming (live):** Flink job consumes canon.state_event / production_count topics → maintains the open state_interval, a rolling Redis OEE for the current shift, and micro-stop/speed counters for Monitoring. Latency within D2 (minutes).
- **Batch (mart):** Airflow DAG closes intervals, computes oee_interval per shift/day, runs loss_attribution, builds top_loss, refreshes readiness_snapshot. The mart is the system of record for reporting; the stream is for the live tile.

Both compute from the **same canonical events and the same definitions** — the live tile and the shift report cannot disagree beyond in-flight events.

2. Pipeline stages

flowchart LR

```

EV["canon.state_event / downtime_event / production_count"] --> RES["State Interval Resolver (idempotent)"]
RES --> CALC["OEE Calc (A×P×Q + TEEP)"]
CALC --> LOSS["Loss Attribution (Six Big Losses)"]
LOSS --> PAR["Top-Loss Pareto"]
CALC --> PUB1["Publish: KPI registry + Redis live + canonical write-back"]
LOSS --> PUB2["Publish: KPI registry + Redis live + canonical write-back"]
PAR --> PUB3["Publish: KPI registry + Redis live + canonical write-back"]

```

3. Recompute & replay (NFR)

Any correction upstream (a re-coded reason, a fixed timestamp, a recalibrated ideal cycle) triggers a **bounded recompute** of only the affected (work_unit, time-bucket) partitions via idempotent upsert (unique key on oee_interval). No manual patching; OEE history is always a deterministic function of canonical events + effective-dated policy. A nightly **reconciliation job** asserts M4 totals vs M2 downtime-minutes and M3 counts (spec 06).

4. Domain events M4 publishes

Event	When	Consumers
oee.interval.computed	shift/day rollup done	UIL, Reporting
oee.state.changed	live state transition resolved	Monitoring (live tile)
oee.microstop.detected	sub-threshold stop (auto path)	Monitoring, M2 (bad-actor)
oee.speedloss.flagged	sustained sub-rate running	Monitoring, M3 (re-plan),

oee.toploss.updated	Pareto recomputed	Financial
oee.other_bucket.exceeded	"All Other" entered top loss	UIL, Reporting
		CI engineer / config

5. Acceptance (MUST)

- P-01 Live (stream) and mart (batch) OEE for a closed shift agree to the unit.
- P-02 A late/corrected event recomputes only affected buckets and changes the result deterministically.
- P-03 Nightly reconciliation passes: Σ M4 unplanned-downtime-min = Σ M2 breakdown-min for the shared events; Σ M4 total_count = Σ M3 ProductionCount.
- P-04 Every published event is idempotent (replay-safe) and tenant-scoped.

M4 · 06 · KPIs, Analytics & Financial Impact

Deep-plan: \$9, \$12 · **Build phase:** M4-2→3

1. KPI family (into the shared registry, doc 03)

KPI

OEE

Availability

Performance

Quality

TEEP

Utilisation/Loading

Six-Big-Losses split

Top-loss Pareto

Planned/Unplanned downtime

All definitions live in the **shared ISO 22400 KPI registry** — M4 publishes values, never a private definition. UIL/Reporting/M2/M3 read the same registry.

2. The MTBF/MTTR boundary (reliability is M2)

M4 does **not** publish MTBF/MTTR. Those are M2's reliability/maintainability KPIs over the *same* DowntimeEvent stream. M4 publishes **time-based Availability** and breakdown-loss minutes; M2 reads those events for failure stats. One downtime truth, two views (deep-plan \$12). Reviewers: any MTBF/MTTR in oee.* is a defect.

3. Financial impact (D7, rate-as-of-event)

Loss	Priced as	Rate
Availability (downtime)	lost-min × downtime	cost_rate (downtime)

	cost/min	effective at event date
Performance (speed/micro-stop)	lost-units × lost-margin/unit	cost_rate (throughput)
Quality (defect/yield)	scrap+rework units × cost/unit	cost_rate (scrap)

Resolution **always** uses the rate **effective at the event date**, never today's (D7).
 loss_attribution.lost_cost + cost_rate_ref persist the provenance.

4. Reconciliation (the trust test)

A nightly job asserts, per shift × work-unit: - Σ oee.loss_attribution(L1+L2).lost_minutes = Σ M2 downtime minutes for the shared downtime_event_ref; - oee_interval.total_count = Σ M3 ProductionCount.total for the shift; - oee_interval.good_count = Σ ProductionCount.good. Any drift raises an alert — this is what operationally guarantees “maintenance, production, quality quote one OEE.”

5. Acceptance (MUST)

- K-01 Registry values for A/P/Q/OEE/TEEP equal oee_interval to 4 dp.
- K-02 No MTBF/MTTR is published by M4.
- K-03 Loss pricing uses the event-date cost_rate; changing today's rate does not change a historical loss cost.
- K-04 Reconciliation vs M2 (minutes) and M3 (counts) passes on the fixture; drift alerts fire when seeded.

M4 · 07 · APIs, Events & Integration

Deep-plan: \$0, \$5, \$12, \$14 · **Build phase:** M4-1→4

1. REST (OpenAPI, OIDC, tenant-scoped)

Method · Path	Purpose
GET /oee/assets/{id}/current	live OEE + current state (Redis)
GET /oee/assets/{id}/intervals? grain=shift&from=&to=	OEE interval series (mart)
GET /oee/assets/{id}/losses?period=	loss tree + top-loss Pareto (minutes & cost)
GET /oee/rollup?scope=line plant&from=&to=	summed-quantity rollup (no avg)
PUT /oee/config · PUT /oee/assets/{id}/config	policy: threshold, planned-time convention, fidelity
PUT /oee/ideal-cycle	upsert ideal cycle (material × work-unit), versioned
GET /oee/assets/{id}/readiness	Required/Quality/Fidelity/History gates
POST /oee/recompute	bounded replay for a (asset, window)

All reads served from the Serving zone/mart; writes are policy/reference only (M4 never writes canonical events).

2. Domain events

Publishes the spec-05 §4 set (oee.interval.computed, oee.state.changed, oee.microstop.detected, oee.speedloss.flagged, oee.toploss.updated, oee.other_bucket.exceeded). Consumes canon.state_event, canon.downtime_event, canon.production_count, plus m2.* failure classification and m3.* plan-target updates.

3. Manifold ingest on-ramp (historian)

For clients with an existing OEE/MES historian, Manifold maps states/reasons/counts onto the canonical contract via the **OEE sector template** (steel cold-rolling seeded): state map, reason→loss→component map, ideal-cycle table, count map. No M4 code change — ingestion is a Manifold mapping (doc 02).

4. Edge collector contract (FUTURE — switch-on, D9)

Reserved interface for automatic capture (spec 03 §4): an edge gateway client (OPC-UA / MQTT / digital-IO / current-clamp) emits canonical state_events after **edge-side debounce + micro-stop threshold**. Contract: {tenant_id, work_unit_ref, signal_ts, running:bool, source_tag} → resolver. Built when a client connects PLCs; not in v1.

5. MVP migration bridge (target design + migration)

The existing OEE/Downtime **MVP** is the head-start; migration is **non-destructive and incremental**: | MVP today | Target | Migration step | |—|—|—| | OEE from own tables/direct feed | OEE from canon.* Event Spine | repoint calc engine at the Serving API; keep MVP UI as a consumer | | flat/ad-hoc reasons | 3-layer tree → loss → component | import reasons as L1/L2 leaves; backfill loss_category_ref/oee_component; keep old codes as aliases | | hard-coded ideal cycle | versioned oee.ideal_cycle | seed from the MVP constant; flag uncalibrated | | A/P/Q in app code | shared ISO 22400 registry | move definitions to registry; reconcile | **First acceptance gate**: MVP OEE and canonical OEE **agree to the unit on a back-test window** (tolerance per open decision, e.g. ≤0.5 pts/30 days) before the MVP is repointed. No history discarded; reasons mapped forward.

6. Acceptance (MUST)

- A-01 All endpoints OIDC-secured and tenant-scoped (D8); reads never cross tenants.
- A-02 No endpoint writes a canonical event; only oee.* policy/reference + derived rollups.
- A-03 Historian ingest works via Manifold mapping with zero M4 code change.
- A-04 Edge-collector contract is documented and stub-tested (no v1 build).
- A-05 MVP back-test reconciliation passes within tolerance before repoint; old reason codes resolve via aliases.

M4 • 08 • Prediction / ML — Future-Scope Interfaces

Deep-plan: §8 • **Build phase:** M4-5 (GATED on data + D9) • **No v1 build**

1. Stance

Mirrors M2 (predictive) and M3 (optimisation): **future-proof now, build later, gated on accumulated history**. v1 ships the data substrate and interface stubs only. No prediction is promised in v1 — only prediction *readiness*.

2. Capabilities (when built)

Capability	Input substrate (built in v1)	Output
Breakdown / micro-stop risk	labelled state_interval + reason codes	probability per asset/window → M2 PdM
Speed-loss degradation forecast	calibrated ideal-cycle + Performance series	predicted slow-creep before it costs a shift
Quality-loss window prediction	startup/transition loss_attribution	likely L6 windows (grade change, run-up)
Prescriptive	all of the above + cost	“act now to protect Availability” (2–3 yr)

3. Shared substrate with M2

M4 loss prediction and M2 predictive maintenance converge on the **same DowntimeEvent stream + OSA-CBM pipeline**. The breakdown-risk model is a **shared platform asset**, not a duplicate — M4 contributes the OEE-loss labels, M2 the condition/failure features. Build coordinates with M2 spec 08.

4. Readiness gate (the only v1 deliverable here)

oee.readiness_snapshot carries a **HISTORY** gate per asset: enough labelled loss history (threshold = open decision; shared with M2’s PdM gate) flips prediction_enabled eligibility. Until then the UI shows “prediction-ready: not yet — N weeks of labelled loss history needed,” never a fake forecast.

5. Stub

oee.loss_prediction exists, empty, unwritten in v1. Populated only when models are built and the HISTORY gate passes.

6. Acceptance (MUST)

- F-01 No v1 code path writes oee.loss_prediction.
- F-02 The HISTORY gate computes and displays per-asset readiness honestly.
- F-03 Interfaces are documented so the model build is a switch-on, not a re-model.

M4 · 09 · Security, RBAC, NFRs & Acceptance

Deep-plan: §13 · Cross-cutting

1. RBAC (extends M1: Operator / Supervisor / Plant-Head / Admin)

Role	Can
Operator	view live OEE tile for own line; reason-code stops
Shift Supervisor	shift OEE, loss Pareto; validate reason coding
OEE / CI Engineer	configure state model, micro-stop thresholds, ideal-cycle, reason tree, targets; run loss analysis
Maintenance/Reliability	read breakdown losses (shared with M2)
Plant Manager	OEE/TEEP trends, cost of loss (read-only analytics)
Admin	masters, integration, capture-fidelity & OEE policy, posture of recompute

Row-level scoping by site/area (inherits M1 line_access). OIDC via Keycloak.

2. Definition-change governance (OEE-specific)

Because ideal cycle, micro-stop threshold, and planned-time convention **move the number**, every change is **effective-dated and audit-logged** (oee.policy_change_log) and surfaced on trends (“definition changed here”). A trend can never be silently broken by a re-definition.

3. NFRs (delta over platform §7)

Concern	Target
Latency	live tile reflects a new state within D2 budget (minutes, tunable)
Determinism	resolver + calc idempotent & replayable ; same events ⇒ same OEE
Reconciliation	M4 totals tie to M2 minutes and M3 counts nightly (spec 06 §4)
Confidence	every OEE carries a fidelity flag (MANUAL/HISTORIAN/AUTOMATIC)
Offline	first-party/edge capture queues locally, syncs on reconnect (like M1)
Multi-tenant	D8 logical isolation; tenant_id everywhere
Hybrid	resolver + tile + collector run on-prem edge (D1)

4. Acceptance (MUST / SHOULD)

- **MUST** A/P/Q/OEE/TEEP match ISO 22400 worked examples (spec 02 C-01..05).
- **MUST** rollups sum-then-divide; no averaging of percentages (C-02).
- **MUST** loss leaves reconcile to factor gaps (L-02) and to M2/M3 (K-04, P-03).
- **MUST** one downtime_event_ref shared with M2 — no double count (M-05).
- **MUST** definition changes effective-dated & audit-logged; recompute deterministic.
- **MUST** MVP back-test reconciles within tolerance before repoint (A-05).
- **MUST** no v1 write to oee.loss_prediction; HISTORY gate honest (F-01..02).
- **SHOULD** auto-capture replay reproduces Availability within tolerance & sets micro_stops_observed (S-05).
- **SHOULD** “All Other” governance prompt fires above threshold (L-04).

5. Test plan

Unit: factor math, time-model, threshold classification, pricing rate-as-of. Integration: M1-bootstrap — OEE; historian ingest; recompute/replay. Reconciliation: nightly M4→M2→M3. Fixtures: a Hero-Steels shift (manual) + a discrete-part shift (clean cycle) for cross-sector (D11).

M4 • 10 • Glossary

OEE & loss terms used across the M4 handover.

- **OEE** — Overall Equipment Effectiveness = Availability × Performance × Quality (ISO 22400-2). % of Planned Production Time spent making good product at full speed.
- **Availability** — Operating Time ÷ Planned Production Time. Loses breakdowns (L1) + setup (L2).
- **Performance** — (Ideal Cycle Time × Total Count) ÷ Operating Time. Loses minor stops (L3) + reduced speed (L4).
- **Quality** — Good Count ÷ Total Count. Loses defects (L5) + startup/yield (L6).
- **Planned Production Time (PPT)** — scheduled time minus planned non-production; the OEE denominator; defined once by canon.shift.
- **Operating Time** — PPT minus all downtime (the running time).
- **Calendar Time** — 24×7 wall clock; the TEEP denominator.
- **TEEP** — Total Effective Equipment Performance = OEE × Utilisation (PPT ÷ Calendar). Exposes unscheduled capacity.
- **OPE / OAE** — OEE variants where “ideal” is a demonstrated-best rate rather than theoretical nameplate.
- **Six Big Losses** — L1 Breakdown, L2 Setup & adjustment, L3 Idling & minor stops, L4 Reduced speed, L5 Process defects, L6 Startup/yield (Nakajima/TPM).
- **Loss tree** — top-down decomposition of the OEE gap into the Six Big Losses by reason; M4’s actual product.

- **Ideal Cycle Time / Rated Speed** — the theoretical fastest time per unit (or rate); the Performance denominator; keyed material × work-unit, versioned.
- **Micro-stop / minor stop** — a stop below the micro-stop threshold (60–120 s); a Performance loss (L3), not Availability.
- **Micro-stop threshold** — the configurable seam between Availability (above) and Performance (below); the most under-measured boundary in OEE.
- **Debounce** — collapsing signal flapping so a single real stop is not logged as many.
- **State model** — controlled vocabulary of machine states, each with `is_planned` and `counts_as` (OEE component).
- **State interval** — a closed, debounced span of one state; the time base for Availability.
- **Capture fidelity** — MANUAL (M1 operator), HISTORIAN (ingested), or AUTOMATIC (PLC/OPC-UA); drives the confidence shown beside OEE.
- **Reason code** — operator-selected cause of a stop; 3-layer tree (category → subsystem → root cause); carries loss category + OEE component.
- **“All Other” bucket** — the single unclassified reason; governed so it never hides a top loss.
- **Top-loss Pareto** — ranked losses by minutes **and** by cost (often different events).
- **DowntimeEvent / StateEvent / ProductionCount** — canonical Event-Spine rows M4 aggregates; **shared** with M2 (failure), M3 (attainment), M5/M6 (yield/quality) — recorded once, never double-counted.
- **MTBF / MTTR** — mean time between failures / mean time to repair; **M2’s** reliability KPIs on the same downtime stream — *not* published by M4.
- **Fidelity gate / History gate** — readiness sub-gates: is capture trustworthy enough to show a confident OEE; is there enough labelled loss history to train prediction.
- **Planned-time convention** — whether planned stops leave PPT or sit inside Availability; a per-tenant ISO-valid modelling choice.